Development Guide

Overview

This guide describes how to set up a development environment and extend the pipeline

Repository Structure

```
src/
  adapters/
    cli/
      main.py                 # CLI dispatcher
      pipeline_full.py        # Full pipeline execution logic (run_full)
      check_environment.py    # Environment diagnostic tool
      integration/            # Stage scripts
  domain/                     # Core data models and logic
  scripts/                    # Utility and conversion scripts
data/
  integration/
    runs/                     # Versioned outputs (git-ignored)
docs/                         # MkDocs documentation
tests/                        # Unit and integration tests
```

Adding or modifying stages

Overview

The pipeline is composed of **stages**, each responsible for a major part of the integration workflow — such as blocking, enrichment, disambiguation, or merging.
Within a stage, there may exist multiple **use cases**, which represent different operational variants of that stage.

For example:

- A stage like *Human integration* might have two use cases:
  - Incorporate a single human annotation immediately after it's submitted.
  - Incorporate all annotations in bulk once a review round is complete.
- A *Disambiguation* stage might expose separate use cases for heuristic-only runs and model-assisted runs.

This structure allows new functionality to be added incrementally without rewriting existing code.

How code is organized

CLI adapter → Use case → Application services → Database/API adapters

Layer	Purpose	Typical location	Notes

Layer	Purpose	Typical location	Notes
CLI adapters	Entry points that handle arguments, environment loading, and I/O paths.	`src/adapters/cli/integration/`	Thin setup layer; no logic beyond parsing and orchestration.
Use cases	Define *a specific workflow* for a stage. Coordinate services and domain models to achieve one operational goal.	`src/domain/use_cases/`	Each use case encapsulates one scenario (e.g., "merge entries," "update one annotation").
Application services	Implement the actual operations and interact with the database or APIs.	`src/domain/services/`	Each service encapsulates a single capability (e.g., conflict detection, metadata enrichment).
Infrastructure adapters	Handle external connectivity (MongoDB, APIs, file system).	`src/infrastructure/`	Include the database adapter and API clients.

> All services in this project are **application services** — they depend on the `DatabaseAdapter` or other clients.
> This is by design, as each stage interacts with stored metadata or external APIs.

What a use case is

A **use case** represents *one way the system performs a task*.
It defines *how* and *when* services are combined to fulfill a specific need.

You can think of it as a "recipe" built from reusable ingredients (services, models, adapters).

- A **stage** groups multiple related use cases (e.g., all operations related to disambiguation).
- A **use case** is a single executable scenario within that stage.

Each use case:

- Coordinates one or more services.
- Defines input and output parameters explicitly.

- Returns results (counts, file paths, summaries) instead of printing.
- Avoids any direct handling of environment variables or CLI logic.

Adding a new stage or use case

1. **Create a new use case**
    - Add a file under `src/domain/use_cases/<stage_or_scenario>.py`.
    - Define a single, self-contained workflow.
    - Accept all dependencies (e.g., `db_adapter`, `api_client`, `path_in`, `path_out`) as arguments.
    - Return explicit results (counts, summaries, file paths).
    - Do not read environment variables or parse CLI arguments here.
2. **Add or reuse services**
    - Place reusable operations in `src/domain/services/`.
    - Each service handles one focused task and can interact with the database or APIs.
    - If logic is reused across multiple use cases, extract it into a standalone service.
3. **Use domain models**
    - Import models from `src/domain/models/` to represent entities (software entries, conflicts, annotations, etc.).
    - Domain models describe structure and meaning; they ensure consistency across stages.
    - Prefer models to raw dictionaries when possible.
4. **Integrate with the database and APIs**
    - Use the `DatabaseAdapter` (Mongo implementation by default) for persistence.
    - Use API clients under `src/infrastructure/` for web services (Observatory, Europe PMC, Semantic Scholar, etc.).
    - Do not embed connection logic or `requests` calls inside use cases — those belong in services or clients.
5. **Create or extend a CLI adapter**
    - Add a file under `src/adapters/cli/integration/<stage>.py` or extend an existing one.
    - Parse arguments (`argparse`), load environment variables, and instantiate clients.
    - Call the desired use case.
    - Print or save results, handling errors gracefully.
6. **(Optional) Add to the full pipeline**
    - If the new use case should be part of the full integration flow, add a `_run([...])` command in `src/adapters/cli/pipeline_full.py` at the correct step.
    - Use versioned paths (e.g., `<run_id>`) to store intermediate outputs.
7. **Document and test**
    - Add the new use case or stage to `docs/pipeline.md` (Stage Summary and Details).

- Update `docs/installation.md` if new environment variables or tokens are required.
- Write at least one unit test for the use case (mocking the database adapter or clients).

About the infrastructure layer and database adapter

- The database interface lives in `src/infrastructure/db/adapter.py` and defines a `DatabaseAdapter` protocol.
- The current backend is **MongoDB**, implemented by a concrete adapter that follows this protocol.
- Use cases and services depend on this abstract interface, not the concrete Mongo client.

To add or replace the database:

- Implement a new adapter in `src/infrastructure/db/<backend>.py` using the same method signatures.
- Keep return types and behavior consistent.
- Instantiate the new adapter in the CLI if needed — the rest of the pipeline can remain unchanged.

Key principles for contributors

- Use cases define **workflows** (they orchestrate how things happen).
- Services define **capabilities** (they perform the actual operations).
- Adapters define **connections** (they make services usable with real data sources).
- Keep use cases small and explicit — one purpose per file.
- Avoid global state or environment lookups inside use cases.
- Return structured results; let the CLI handle printing, logging, or writing files.
- Always favor **clarity and explicitness** over abstraction: one file, one responsibility, one direction of dependency.

Testing

To run tests, go to the root directory of this repository and use:

```
PYTHONPATH=$(pwd) pytest -v -s tests/
```

The previous command will run all tests except the ones marked as "manual". To run tests marked as "manual" use:

```
PYTHONPATH=$(pwd) pytest -v -s -m manual tests/
```

Logging

To add loggings, use:

```

```

```
import logging

logger = logging.getLogger("rs-etl-pipeline")
```

The logger configuration can be found in `src/infrastructure/logging_config.py`. `INFO` logs are writen to terminal and all the rest to a file (`re_etl_pipeline.log`)